

Web Services Tutorial

Orbix E2A Web Services Integration Platform, Orbix E2A XMLBus, IONA Administrator, and Orbix E2A Application Server are trademarks of IONA Technologies PLC.

While the information in this publication is believed to be accurate, IONA Technologies PLC makes no warranty of any kind to this material including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. IONA Technologies PLC shall not be liable for errors contained herein, or for incidental or consequential damages in connection with the furnishing, performance or use of this material.

This product includes the JMX(TM) Technology. JMX and all JMX based trademarks and logos are trademarks or registered trademarks of Sun Microsystems, Inc. in the U.S. and other countries. Java is a trademark of Sun Microsystems, Inc.

This product includes software developed by the Apache Software Foundation (<http://www.apache.org/>).

This product includes software copyright © 2001, International Business Machines Corporation and others. All rights reserved.

UDDI4J source code is available from IONA as required in section 3(iv) of the IBM license agreement. Contact xmlbus-info@iona.com for more details.

COPYRIGHT NOTICE

No part of this publication may be reproduced, stored in a retrieval system or transmitted, in any form or by any means, photocopying, recording or otherwise, without prior written consent of IONA Technologies PLC. No third party intellectual property right liability is assumed with respect to the use of the information contained herein. IONA Technologies PLC assumes no responsibility for errors or omissions contained in this book. This publication and features described herein are subject to change without notice.

Copyright © 2001 IONA Technologies PLC. All rights reserved.

All products or services mentioned in this manual are covered by the trademarks, service marks, or product names as designated by the companies who market those products.

M 2 7 6 7

Contents

Chapter 1	Overview	1
	The Temperature Converter Application	2
	Preparing the Application	3
Chapter 2	Building a Web Service From a Java Application	5
	Building a Web Service From a JAR	7
	Selecting the Web Service's Implementation	11
	Selecting the Web Service's Methods	12
	Including Supporting Classes	14
	Defining WSDL Information	17
	Deploying a Web Service	18
	Generating a Proxy Client	20
Chapter 3	Listing Web Services Using the Web Services Manager	23
Chapter 4	Compiling and Running a Generated Proxy Client	29
Chapter 5	Review	33
	Index	35

Table of Contents

1

Overview

This tutorial describes the typical steps to transform an existing Java application to a Web service using XMLBus. XMLBus provides the tools to build, deploy, manage, and use a Web service.

Note: To use this tutorial effectively, XMLBus must be installed on your system. To download XMLBus, see <http://www.xmlbus.com/work/>.

This tutorial takes you through a complete example, so you learn the processes of creating a Web service. This includes how to use the key tools and what to supply to them, what components are used where in the development process, and what files and information are created. The tutorial includes the following phases:

1. [“Building a Web Service From a Java Application” on page 5](#)
2. [“Listing Web Services Using the Web Services Manager” on page 23](#)
3. [“Compiling and Running a Generated Proxy Client” on page 29](#)

However, before you start the tutorial, you might want to review the basic application code in the next section.

The Temperature Converter Application

This tutorial uses a very simple temperature converter application. For your reference, the example code is shown here:

```
package com.iona.xmlbus.examples;
import java.io.*;

public class TemperatureConverter
{
    public TemperatureConverter()
    {}

    private float centigrade;
    private float fahrenheit;

1    public long getFahrenheit(float centigrade){
        this.centigrade = centigrade;
        float result_1 = centigrade * 9;
        float result_2 = result_1 / 5;
        float final_answer = result_2 + 32;
        Float temp = new Float(final_answer);

        return temp.longValue();
    }

2    public long getCentigrade(float fahrenheit){
        this.fahrenheit = fahrenheit;
        float result_1 = fahrenheit - 32;
        float result_2 = result_1 / 9;
        float final_answer = result_2 * 5;
        Float temp = new Float(final_answer);

        return temp.longValue();
    }
}
```

The temperature converter application consists of two methods:

1. `getFahrenheit()` takes a float value representing a Centigrade temperature and returns a long representing the Fahrenheit equivalent.

2. `getCentigrade()` takes a `float` value representing a Fahrenheit temperature and returns a `long` representing the Centigrade equivalent.

Preparing the Application

Note: If the file `converter.jar` is available in your installation (See *XMLBus-installation/XMLBus/demos/TemperatureConverter/Lib/*), skip this section and go to [“Building a Web Service From a Java Application” on page 5](#).

If the application's Java, class, and JAR files are not available on your system, prepare the application as follows:

1. Create a `TemperatureConverter.java` file in a working directory by copying the code shown in the section [“The Temperature Converter Application” on page 2](#).

Note: Be sure the callout numbers used to describe the code are not left in.

2. Create a `TemperatureConverterTest.java` file in the same working directory by copying the code shown here:

```
package com.iona.xmlbus.examples;
import java.io.*;

public class TemperatureConverterTest
{
    public static void main(String args[]){
        try{
            byte[] b = new byte[1000];

            System.out.println("Please Enter a Number: \n");
            int length = System.in.read(b);

            ByteArrayOutputStream bout = new
            ByteArrayOutputStream();
            bout.write(b);
```

```
        String request = bout.toString();
        Float floatObj = new Float(request);

        TemperatureConverter converter = new
TemperatureConverter();

        long fTemp =
converter.getFahrenheit(floatObj.floatValue());
        System.out.println(request.trim()+" "+" Degrees
Centigrade converted to Fahrenheit is:  "+ fTemp +"\n");

        long cTemp =
converter.getCentigrade(floatObj.floatValue());
        System.out.println(request.trim()+" "+" Degrees
Fahrenheit converted to Centegrade is:  "+ cTemp);

    }catch(IOException io)
    {
        io.printStackTrace();
    }
}
```

3. Run the Java compiler to create the class files.

```
javac -d *
```

4. Create a Java Archive file (JAR), converter.jar, using the **jar** command:

```
jar cvf converter.jar TemperatureConverter.class
TemperatureConverterTest.class
```

To get started with the tutorial, go to [“Building a Web Service From a Java Application” on page 5](#).

2

Building a Web Service From a Java Application

The XMLBus Web Service Builder is a graphical tool that walks you through the steps to make your application a Web service. This tool creates all the files needed for a Web service, based on an application you specify and other information you input. This tutorial uses an existing Java application with one Java class.

Figure 1 shows the input required and the typical output produced for the XMLBus Web Service Builder when transforming a Java application to a Web service.

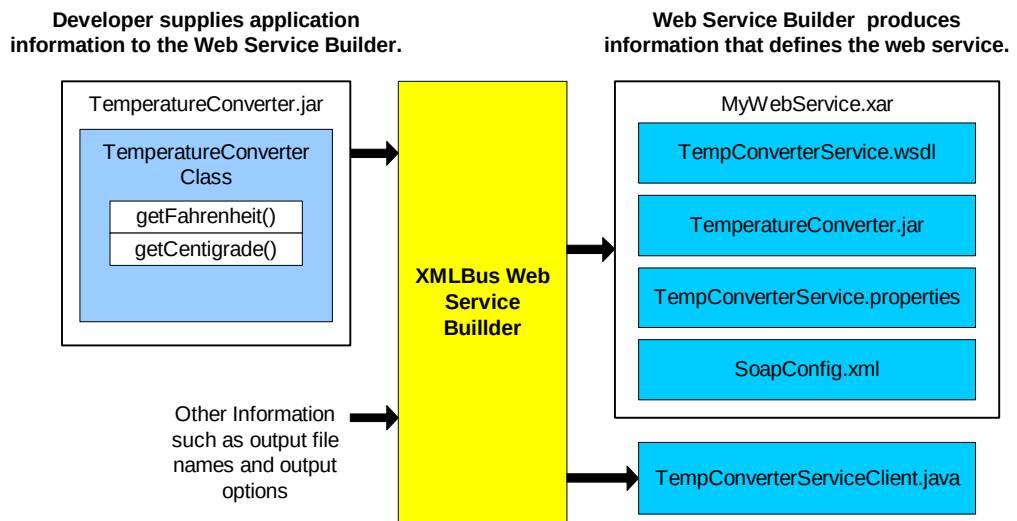


Figure 1: *Input and Output for the Web Service Builder*

After you give the builder the location of the application's Java class implementation along with other information required, the builder produces an XMLBus archive file (XAR) and code for a stand-alone client to the Web service.

The XAR contains a WSDL file that describes the Web service, the implementation code in the form of a JAR file, a properties file describing the settings from the Web Service Builder, and a SOAP configuration file describing deployment configuration information that the Web Services Container uses to deploy the Web service.

The following sections describe the steps to build a Web service from a Java application:

1. [“Building a Web Service From a JAR” on page 7](#)
2. [“Selecting the Web Service's Implementation” on page 11](#)
3. [“Selecting the Web Service's Methods” on page 12](#)
4. [“Including Supporting Classes” on page 14](#)
5. [“Defining WSDL Information” on page 17](#)
6. [“Deploying a Web Service” on page 18](#)
7. [“Generating a Proxy Client” on page 20](#)

Building a Web Service From a JAR

1. Start the Web Service Builder from the **Start** menu. For example, **Start|Programs|IONA|XMLBus|Web Service Builder**. Figure 2 shows the tool's opening page.

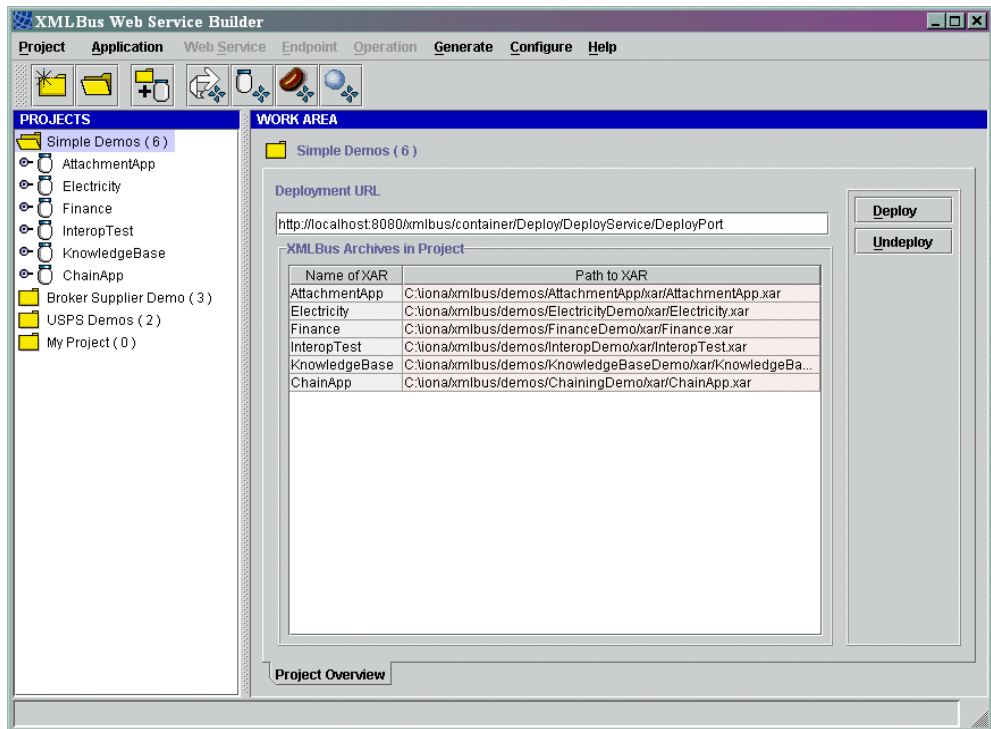


Figure 2: The XMLBus Web Service Builder

2. Select the **Simple Demos** project from the left-most column.

3. From the menu, select **Application|Create Web Service|From Archive**. [Figure 3](#) shows the first of the seven windows that guide you through the creation of a Web service.

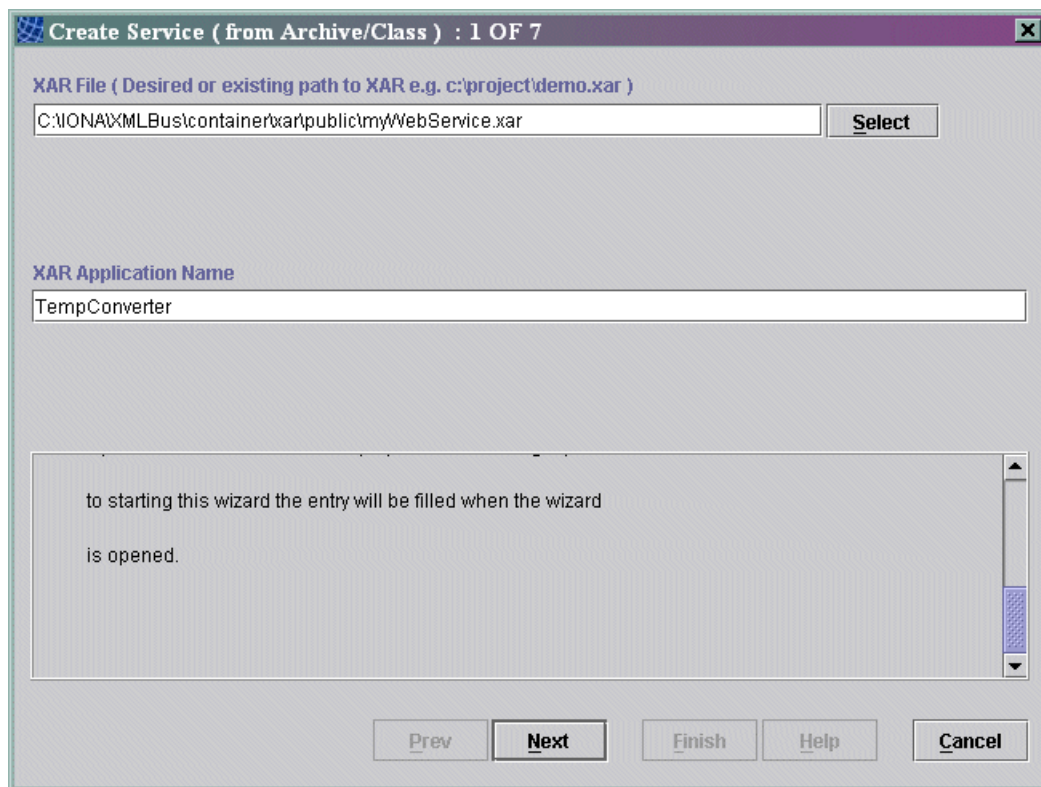


Figure 3: *Naming the Web Service*

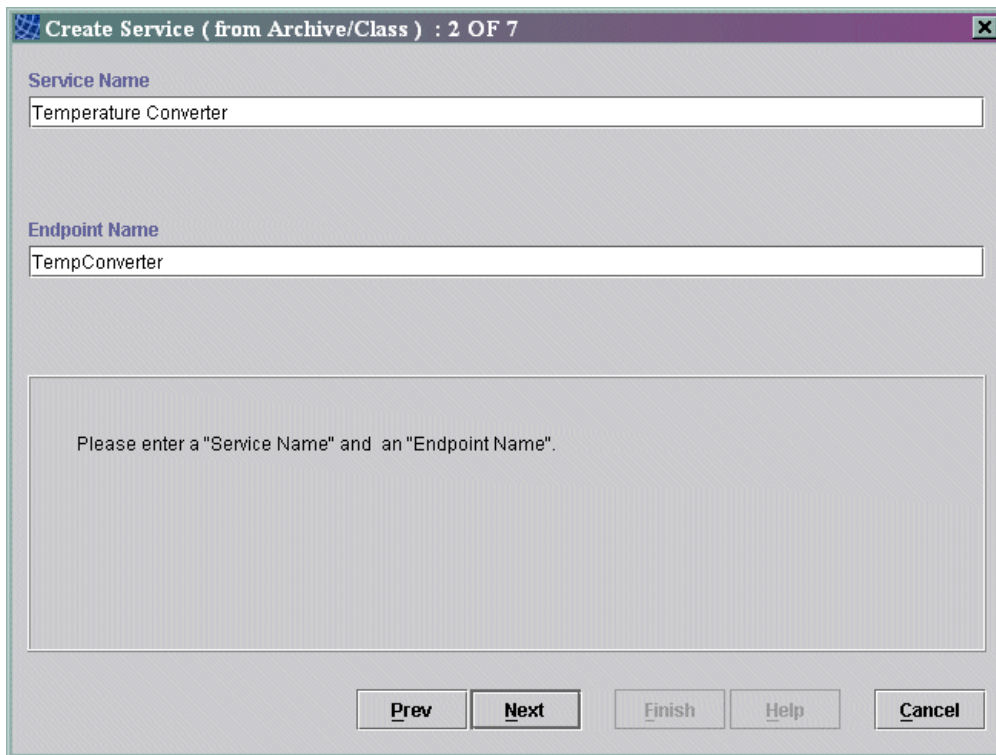
To build a Web service, the Web Service Builder requires input from one of the following sources:

A Java Class	An existing Java application can be transformed into a Web service.
A deployed EJB	An existing Enterprise Java Bean application deployed into a J2EE application server can be transformed into a Web service.
An Archive File	An existing XAR, JAR, or Zip archive can be transformed into a Web service.
A CORBA Object	An Existing CORBA Object whose interface is stored in an Interface Repository.

This example uses a Java class from an existing JAR.

4. Enter the **XAR File Name**. For this example, enter **<install_dir>\container\xar\public\myWebService.xar**.
This is where the Web Service Builder stores the information gathered during the creation of a Web service.
5. Enter the **XAR Application Name**. For this example, enter **TempConverter**.
The Web Services Container uses this name to identify the Web service application and distinguish it from other applications that the Web service might use. This example consists of a single application.

6. Select **Next** to display the window shown in [Figure 4](#).



The screenshot shows a Java Swing dialog box titled "Create Service (from Archive/Class) : 2 OF 7". It has a light gray background and a purple title bar. There are two text input fields: "Service Name" with the text "Temperature Converter" and "Endpoint Name" with the text "TempConverter". Below these fields is a large rectangular area containing the text "Please enter a 'Service Name' and an 'Endpoint Name'." At the bottom right, there are five buttons: "Prev", "Next", "Finish", "Help", and "Cancel". The "Next" button is highlighted.

Figure 4: *Naming your Web Service.*

7. Enter a **Service Name** for your Web service. For this example, enter **TempConverterService**.
This is the name that your Web service will be known by to the outside world. It will also denote the Web service in the XMLBus tools.
8. Enter an **Endpoint Name** for your Web service. For this example, enter **TempConverterPort**.
The endpoint name identifies the interface to your Web service.
9. Select **Next** and go to [“Selecting the Web Service’s Implementation” on page 11](#).

Selecting the Web Service's Implementation

1. Select the **Select** button and browse the system directories to find and select the archive to use. For this example, select the **converter.jar** file, which should be located in a demonstration examples directory such as the following:
`xmlbus-installation\demos\TemperatureConverter\lib`
 (If the file is not available, see [“Preparing the Application” on page 3.](#))
2. The archive's list of valid classes are displayed in a window similar to [Figure 5](#). For this example, select the **TemperatureConverter** class.

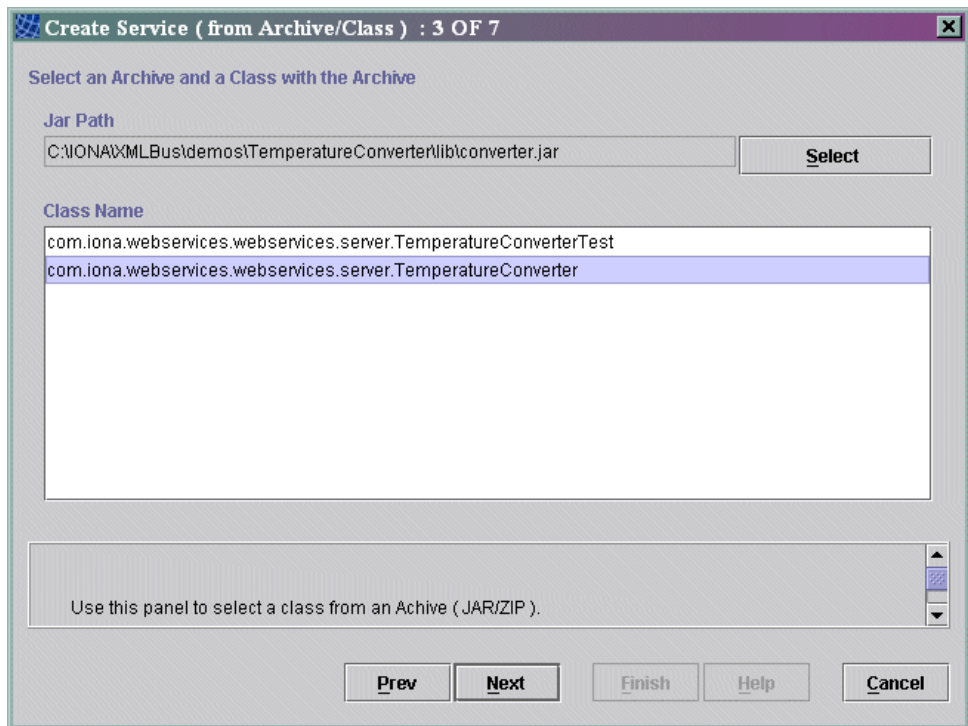


Figure 5: *Selecting a Class from an Archive with the Web Service Builder*

Note: Only classes that use data types supported by XMLBus are displayed even though the archive may contain other classes.

3. Select **Next** and go to [“Selecting the Web Service’s Methods”](#).

Selecting the Web Service’s Methods

After selecting the implementation class, select the methods the Web service will support. A Web service can use all the methods defined in the application’s class, or a subset of those defined in the class. To select which methods to include in your Web service, follow the next set of step.

1. A window similar to [Figure 6](#) lists the methods defined in the selected application. From this list, select the methods to use for your Web service. For this example, select `getFahrenheit()` and `getCentigrade()`.

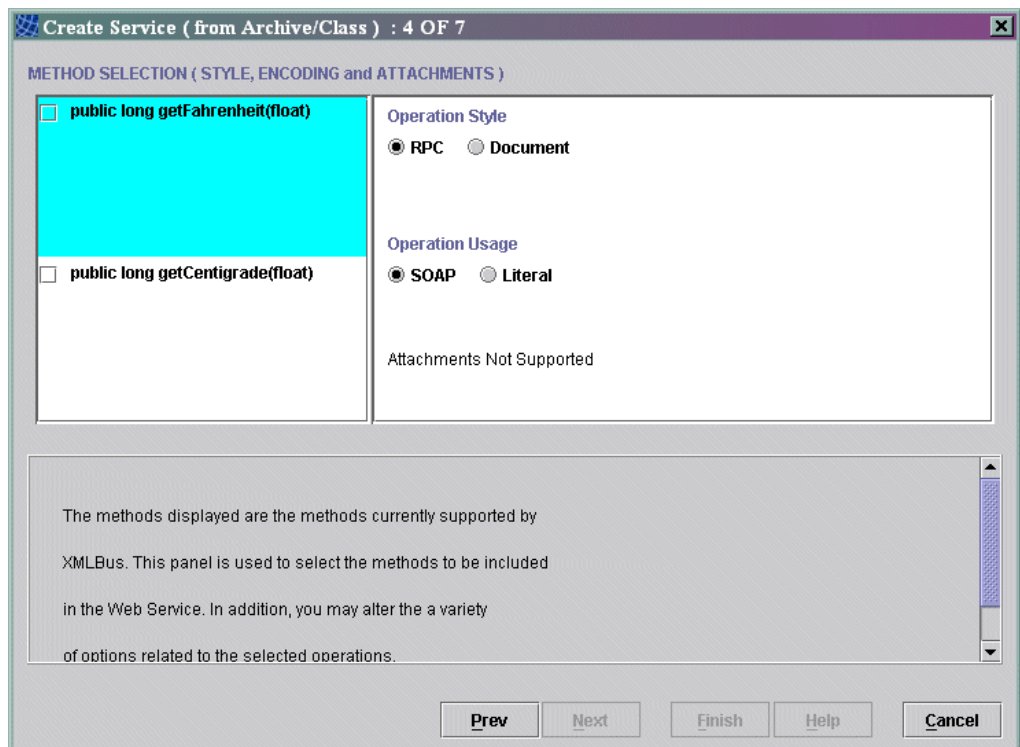


Figure 6: *Selecting Web Service Methods with the Web Service Builder*

Note: Methods that use unsupported data types are displayed in gray.

2. The panel on the right lists the style and encoding to use on the SOAP message. For this example, select, **RPC** and **SOAP**.
3. Select **Next** and go to [“Including Supporting Classes” on page 14](#).

Including Supporting Classes

After selecting the methods the Web service will use, you must identify all the classes that the Web service needs to access. The following steps show how to include the application's classes into the Web service.

- 1. Ensure that the JAR for your application is included in the list of **XAR Classpath Entities**. If it is not, select the **Add** button and use the browser to locate the needed JARs.

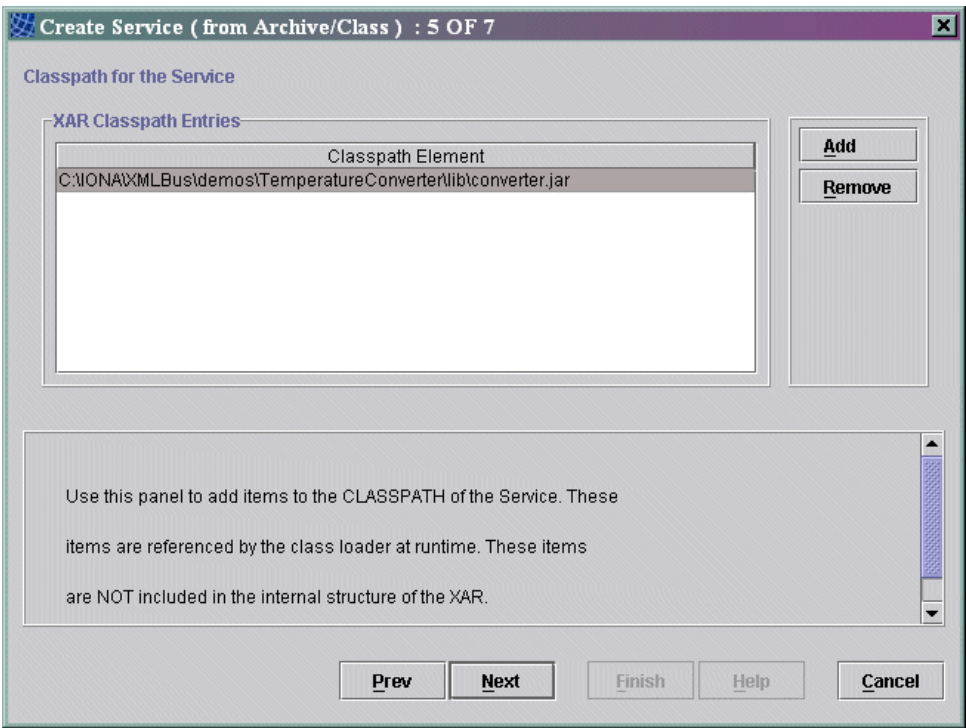


Figure 7: Adding your JAR to the CLASSPATH

If there are no classes listed, complete the following steps:

- i. Select **Add a Supporting Class**.
- ii. Browse the system directories to find and select **converter.jar**, which should be located in a demos directory such as the following:
xmlbus-installation\demos\TemperatureConverter\lib

Note: The items listed here are added to the Web Service Container's CLASSPATH and not included in your Web services XAR.

2. When you have selected all of the entries you want to add to the CLASSPATH, select **Next**.

3. A window similar to [Figure 8](#) should display. This window lists all of the archives and/or classes that will be included in your Web service's XAR. For this example, leave this window empty.

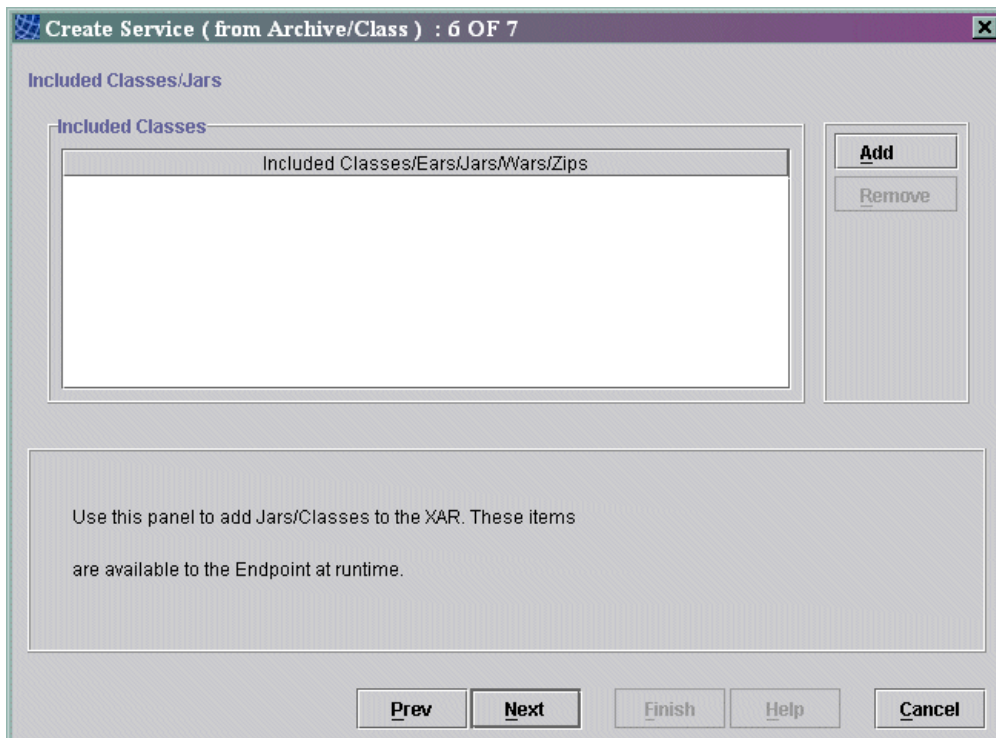


Figure 8: *Selecting Classes to include in a XAR*

4. Select **Next** and go to [“Defining WSDL Information” on page 17](#).
In addition to the implementation class, the application may require other supporting classes. You can repeat the above steps as necessary to add any supporting files required.

Defining WSDL Information

The Web Service Builder generates a Web Service Description Language (WSDL) file to describe the Web service you are building so that applications can find and use it. This file has information such as the implementation's SOAP Endpoint URL and the methods available to the Web service.

The information in the WSDL is generated automatically. However, you may wish to specify the Web service's WSDL namespace or its data namespace, as shown in [Figure 9](#).

Create Service (from Archive/Class) : 7 OF 7

Schema Namespace
urn:TempConverter

Target Namespace
urn:DataTempConverter

Enter the namespace information to be used with for
the WSDL and Complex Types.

Prev **Next** **Finish** **Help** **Cancel**

Figure 9: *Defining WSDL Settings with the Web Service Builder*

Complete the following steps to finish creating your Web service.

1. For this example, use the default value for the **Schema Namespace** used in the WSDL file. The default value displayed is constructed from the name of the Java implementation class input.
If names from multiple WSDL files are likely to conflict, or if the WSDL file defines several services, a namespace uniquely identifies each Web service.
2. For this example, use the default value for the **Target Namespace**. The default value displayed is constructed from the name of the Java implementation class input.
Complex data types such as arrays require the target data namespace.
3. Select **Finish**. The Web Service Builder creates the information needed to deploy your Web service and stores it in the XAR you specified. Once you have returned to the main window, go on to [“Deploying a Web Service” on page 18](#).

Deploying a Web Service

Once you have created a Web service, you need to deploy it so that it is visible to the outside world. The following steps walk you through deploying a Web service using the Web Service Builder.

1. Select **Archive|Deploy**. A window similar to [Figure 10](#) displays.

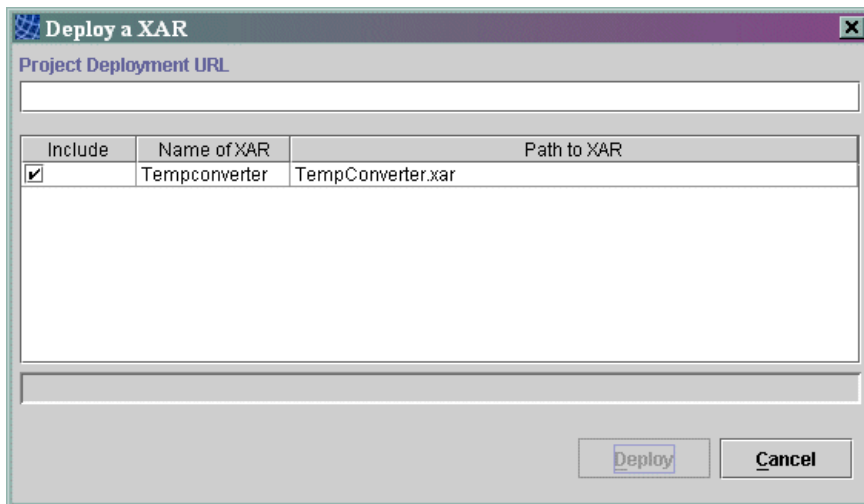


Figure 10: *Specifying Web Services to Deploy*

2. Unselect any XARs you do not wish to deploy by double clicking on them. Only XARs with a check in the include column will be deployed. For this example, only include **TempConverter**.
3. Check the deployment URL and update it to point to the correct end point if need be.
4. When all of the desired XARs are checked off, click **Deploy** and go to [“Generating a Proxy Client” on page 20](#).

Generating a Proxy Client

Once the Web service is deployed, you need a way to access it. The Web Service Builder provides you with the tools to generate two types of client code:

- | | |
|---------------------|--|
| J2ME Client | The Web Service Builder can generate a Java 2 Micro Edition (J2ME) client that can access the Web service. You can compile and run the J2ME Client application to access the Web service's methods. This client has hard-coded values such as the endpoint URL of the Web service. |
| Proxy Client | The Web Service Builder can generate a Proxy Client application. The Proxy Client consists of an interface class that is created at compile time along with an implementation class that is created and instantiated at runtime based on the Java 1.3 proxy scheme. |

For this example, generate a Proxy client.

1. Select **Generate | Proxy Client**. A window similar to [Figure 11](#) should appear.

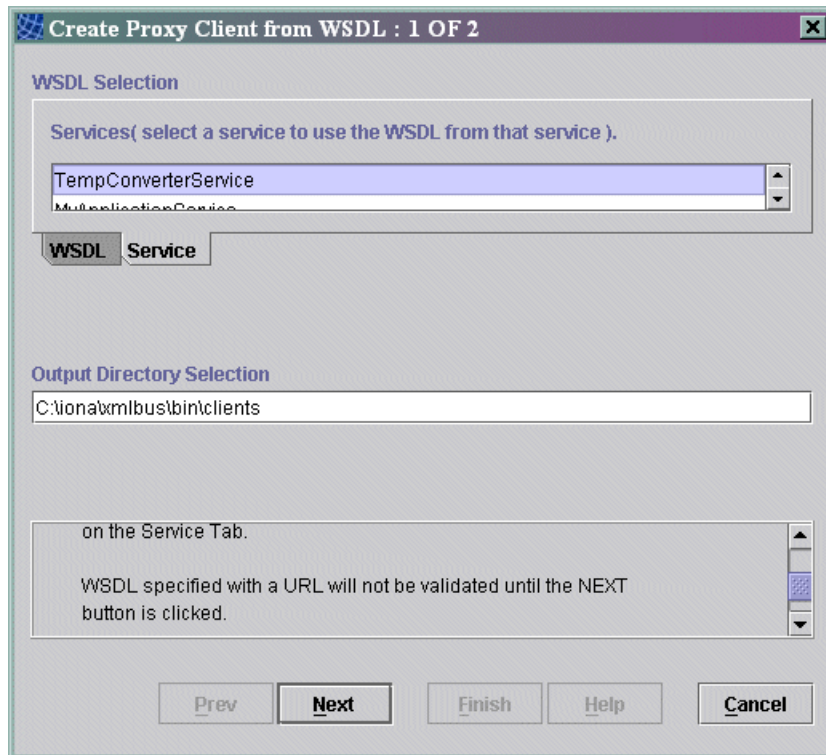


Figure 11: *Generating a Proxy Client*

2. Using the **Service** tab, select the **TempConverterService**.
3. Enter an **Output Directory Selection**. The output directory is where the Web Service Builder puts the generated files. For this example, enter `xmlbus-installation\bin\jar`. Then click **Next**.
4. Using the dropdown lists, select the **Service** and the **Port** to be used in generating the client code. For this example select **TempConverterService** and **TempConverterPort**.
5. Select **Finish**.

The Web Service Builder generates two files:

- ♦ TempConverterProxyDemo.java is the proxy client test program.
- ♦ TempConverterInterface.java is the proxy client interfaces that a developer can use to build a Web service client from.

Go to [“Listing Web Services Using the Web Services Manager” on page 23](#).

3

Listing Web Services Using the Web Services Manager

Use the Web Services Manager tool to view and manage the available Web services your XMLBus installation has deployed. The tool lets you undeploy any Web service, view each Web service's methods, and activate or deactivate a Web service application's endpoints.

Note: Recall that to deploy a Web service you used the **Deploy** command when using the Web Service Builder. (See [“Deploying a Web Service” on page 18](#)).

1. Start the Web Services Manager from a Web browser using the URL from [Table 1](#) that is appropriate for your installation.

Table 1: *Starting the Web Services Manager*

Installation	URL for the Web Services Manager
Stand-alone	http://localhost:8080/xmlbus/
Orbix E2A Application Server	http://localhost:9000/xmlbus/
WebLogic Server	http://localhost:7001/xmlbus/
WebSphere Application Server	http://localhost:9080/xmlbus/

Figure 12 shows the Web Services Manager's opening page.

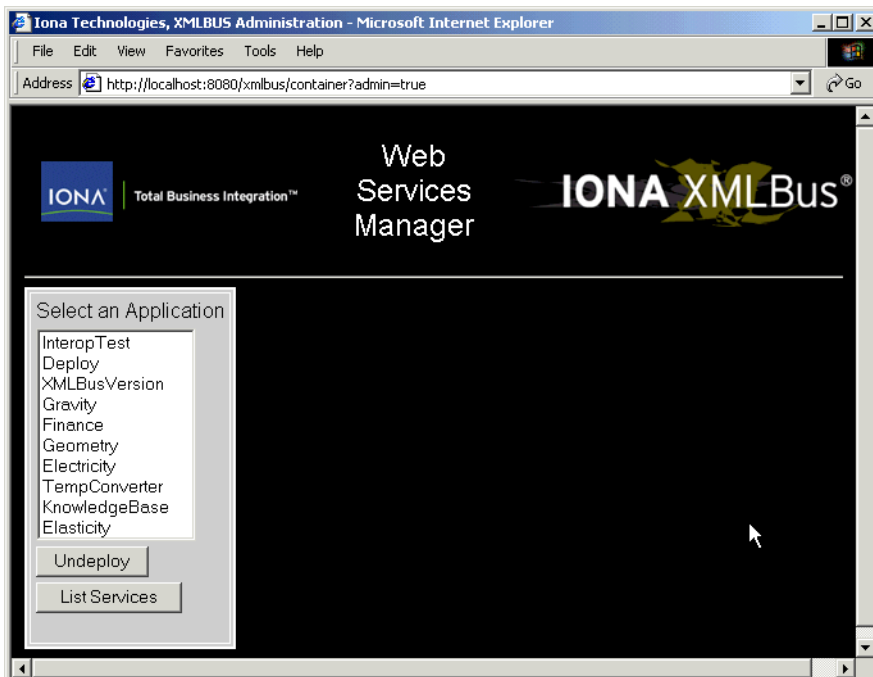


Figure 12: *The XMLBus Web Services Manager*

2. Select the **TempConverter** application.
Options are available to **Undeploy** the Web service application or **List Services** the application provides.

3. Select **List Services** to display the Web service that the TempConverter application provides. Most applications provide a one-to-one mapping between the application and a Web service as shown in [Figure 13](#).



Figure 13: *Listing Web Services with the Web Services Manager*

4. Select the **TempConverterService** service.

5. Select **List Endpoints** to display the endpoint details, as shown in Figure 14.

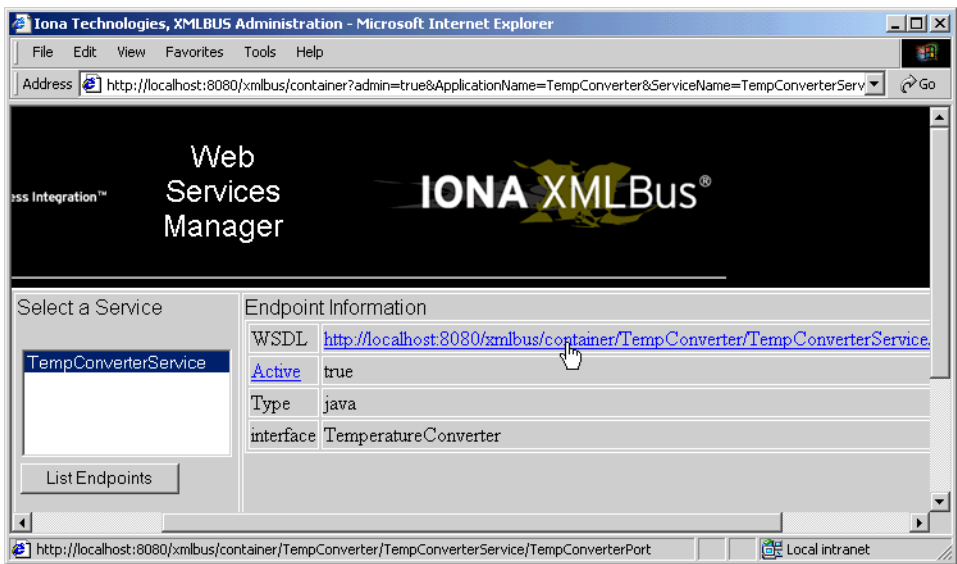


Figure 14: *Endpoint Details with the Web Services Manager*

A Web service's WSDL contains endpoints that describe where the implementation runs on a particular server. Use the Web Services Manager to view and manage a Web service's endpoints.

Endpoint information displayed includes the following:

- WSDL** The full WSDL file URL, which is a selectable link that displays the WSDL file.
- Active** An indicator as to whether the endpoint is active to receive messages. You can select the **Active** link to toggle endpoint activation between active (**true**) and inactive (**false**).
- Type** The language the application is written in.
- Interface** The application's interface name.

-
6. Select the WSDL link to display the WSDL file.

Notice, the location of the endpoint is near the end of the WSDL file as `soap:address location`. For example:

```
<soap:address location="http://localhost:8080/xmlbus/container/TempConverter/TempConverterService/TempConverterPort" />
```

7. Close the WSDL file.

This chapter has described how to list Web services and the associated application endpoint information. Go on to [“Compiling and Running a Generated Proxy Client” on page 29](#) to see how to use the Web service.

4

Compiling and Running a Generated Proxy Client

You can compile and run auto-generated client code to create an application that uses the Web service. (Recall that the Web Service Builder has options to make it generate client code. See [“Generating a Proxy Client” on page 20](#).) You can also incorporate the appropriate portions of the client code into your own applications to seamlessly interact with the Web service.

Note: To compile and run properly, the clients that are generated by the Web Service Builder require certain environment variables. For example, to set environment variables in Windows, select **Start|Settings|Control Panel** and start the **System** program. Look in the **Advanced** tab for the system environment variables.

Follow these steps to compile and run the client:

1. Ensure that your PATH system environment variable includes a valid Java Developer's Kit. For example:
`C:\jdk1.3.1\bin`
2. Ensure that the following JARs are in your CLASSPATH:
`xmlbus-installation\lib\xerces.jar`
`xmlbus-installation\lib\log4j.jar`
`xmlbus-installation\lib\SoapClient.jar`
`xmlbus-installation\devtools\ionaworkbench.jar`
3. Ensure that the directory where the client code is generated is in your CLASSPATH. For example:
`xmlbus-installation\bin\xar`

4. From a system prompt, change to the directory where the client code was generated. For example:

```
cd xmlbus-installation\bin\xar
```

5. Compile the Proxy Client's generated code. For this example, run the following command:

```
javac TempConverterProxyDemo.java  
      TempConverterInterface.java
```

6. Run the Proxy Client demo without arguments to display the usage options and a list of the methods available in the Web service. For example:

```
java TempConverterProxyDemo  
Syntax is: TempConverterProxyDemo [-debug] [-debugparse]  
          [-url soapurl] operation [args...]  
          operation is one of:  
              getFahrenheit  
              getCentigrade
```

7. The following example shows how to run the Proxy Client with an operation and argument.

```
java TempConverterProxyDemo -url http://localhost:8080/  
xmlbus/container/TempConverter/TempConverterService/  
TempConverterPort getFahrenheit 35
```

The previous command example is described as follows:

- **java TempConverterProxyDemo**

This executes the proxy client test application.

- **-url http://localhost:8080/xmlbus/container/TempConverter/TempConverterService/TempConverterPort**

This option represents the endpoint on which the Web service is deployed. The port number 8080 in this example indicates a stand-alone installation of XMLBus. Your port number may be different.

Note: You can obtain this endpoint by using the Web Services Manager. See [Figure 14 on page 26](#). Select from the **Endpoint Information** details, the full URL of the TempConverter WSDL.

- **getFahrenheit 35**

This represents one of the Web service's methods and appropriate arguments. In this example an input value of 35 degrees Centigrade returns the corresponding Fahrenheit value of 95 degrees.

5

Review

In this tutorial you learned how to take an existing Java application class and use the Web Service Builder to generate all the information necessary to transform the application into a Web service.

Building a Web service from a Java application required these steps:

1. [“Building a Web Service From a JAR” on page 7](#)
2. [“Selecting the Web Service’s Implementation” on page 11](#)
3. [“Selecting the Web Service’s Methods” on page 12](#)
4. [“Including Supporting Classes” on page 14](#)
5. [“Defining WSDL Information” on page 17](#)
6. [“Deploying a Web Service” on page 18](#)
7. [“Generating a Proxy Client” on page 20](#)

This tutorial used the Web Service Builder to deploy the Web service. You then learned how to manage the deployed Web service using the Web Services Manager tool. Basic management included the following task:

[“Listing Web Services Using the Web Services Manager” on page 23](#)

This tutorial also used the Web Service Builder to generate a Proxy Client for the Web service. You learned how to access the Web service using this generated client. The developer tasks to use the client are described in the following section:

[“Compiling and Running a Generated Proxy Client” on page 29](#)

Index

A

- archive
 - create service from 8, 9

C

- Class, selecting with the Web Service Builder 11
- Classes, including in the XAR 14
- CLASSPATH environment variable 29
- client code
 - compiling 29
 - generating 20
 - J2ME 20
 - Proxy 20
 - running 29
- Converter application 2
- converter.jar
 - generating 3
 - selecting 11
- CORBA object
 - create service from 9

D

- deploy 18, 19

E

- EJB
 - create service from 9
- endpoint
 - details 26
 - list 26
 - URL 10, 17
- environment variables 29
 - CLASSPATH 29
 - PATH 29
- example application, temperature converter 2

G

- getCentigrade() method 3
- getFahrenheit() method 2

I

- implementation
 - including classes 16
 - language 26
 - referencing classes 14
 - selecting a class from an archive 11
 - selecting for the Web Service 11
 - selecting methods 11, 13
 - selecting operations 12
- Input 5
- input, confirming Web Service Builder 21

J

- J2ME client 20
- Jar
 - create service from 9
- Java 1.3 proxy scheme 20
- Java class
 - create service from 9

P

- PATH environment variable 29
- port numbers 31
- properties file 6
- proxy client 20
 - test client 20

S

- SOAP configuration 6
- SOAP Endpoint URL 10, 17
- SOAP style 13
- source
 - archive file 9
 - CORBA object 9
 - EJB 9
 - Jar 9
 - Java class 9
 - XAR 9
 - Zip file 9
- supporting classes
 - including a reference to 14
 - including in XAR 16

Index

T

Temperature converter application 2

U

undeploy 24

W

Web service

- deploying 18, 19
- listing deployed 25
- selecting classes 11
- selecting implementation 11
- undeploying 24

Web Service Builder 7

- defining WSDL settings 17
- input and output 5
- input, confirming 21
- selecting methods 13
- selecting operations 12
- specifying output 19
- starting 7

Web Service Container 6

Web Service Description Language 17

Web Services Manager 23, 24

- endpoint details with the 26
- listing Web services 25
- starting 23

WSDL 17

- namespace 18

WSDL file 6

- displaying 27

WSDL information, defining 17

WSDL URL 26

X

XAR 6

- application name 9
- create service from 9
- destination file 9
- including files 16
- referencing files 14

XMLBus archive file 6

Z

Zip file

- create service from 9